

# Arquitectura en Capas (Layered Architecture)



## Objetivos

- Entender la arquitectura en capas y sus beneficios
- Conocer las responsabilidades de cada capa
- Aplicar el principio de separación de responsabilidades
- Organizar un proyecto FastAPI en capas

## ¿Qué es la Arquitectura en Capas?

La **arquitectura en capas** organiza el código en grupos horizontales donde cada capa tiene una responsabilidad específica y solo puede comunicarse con las capas adyacentes.

### Principios Fundamentales

1. **Separación de responsabilidades:** Cada capa tiene un propósito único
2. **Dependencia unidireccional:** Las capas superiores dependen de las inferiores, nunca al revés
3. **Abstracción:** Cada capa oculta su implementación interna

## Las Tres Capas Principales

### 1. Presentation Layer (Capa de Presentación)

#### Responsabilidades:

- Recibir requests HTTP
- Validar datos de entrada (Pydantic)
- Serializar respuestas
- Manejar autenticación/autorización HTTP
- Documentación OpenAPI

```
# routers/products.py
from fastapi import APIRouter, Depends, HTTPException, status

from schemas.product import ProductCreate, ProductResponse
from services.product import ProductService
from dependencies import get_product_service

router = APIRouter(prefix="/products", tags=["products"])

@router.post("/", response_model=ProductResponse, status_code=201)
def create_product(
    data: ProductCreate, # Validación automática
    service: ProductService = Depends(get_product_service)
):
    """
    Crea un nuevo producto.

    Esta capa SOLO se encarga de:
    - Recibir el request
```

```

- Validar datos (Pydantic)
- Llamar al service
- Retornar response
"""

try:
    return service.create(data)
except ProductAlreadyExistsError as e:
    raise HTTPException(status_code=409, detail=str(e))

```

## 2. Application Layer (Capa de Aplicación/Servicios)

### Responsabilidades:

- Lógica de negocio
- Orquestación de operaciones
- Validaciones de negocio
- Transacciones (via Unit of Work)

```

# services/product.py
from schemas.product import ProductCreate, ProductUpdate
from repositories.product import ProductRepository
from models.product import Product

class ProductService:
    """
    Capa de servicios: contiene la lógica de NEGOCIO.

    NO conoce HTTP, solo trabaja con objetos de dominio.
    """

    def __init__(self, repo: ProductRepository):
        self.repo = repo

    def create(self, data: ProductCreate) -> Product:
        """
        Crea un producto aplicando reglas de negocio.
        """
        # Regla de negocio: SKU debe ser único
        if self.repo.exists_by_sku(data.sku):
            raise ProductAlreadyExistsError(f"SKU '{data.sku}' already exists")

        # Regla de negocio: precio mínimo
        if data.price < 0.01:
            raise InvalidPriceError("Price must be at least 0.01")

        # Crear entidad
        product = Product(
            name=data.name,
            sku=data.sku,
            price=data.price,
            stock=data.stock
        )

        return self.repo.add(product)

```

### 3. Data Access Layer (Capa de Acceso a Datos)

#### Responsabilidades:

- Persistencia de datos
- Queries a la base de datos
- Mapeo objeto-relacional
- Transacciones de bajo nivel

```
# repositories/product.py
from typing import TypeVar, Generic
from sqlalchemy import select
from sqlalchemy.orm import Session

from models.product import Product

class ProductRepository:
    """
    Capa de datos: acceso a la base de datos.

    NO conoce reglas de negocio, solo CRUD y queries.
    """

    def __init__(self, db: Session):
        self.db = db

    def add(self, product: Product) -> Product:
        """Persiste un producto"""
        self.db.add(product)
        self.db.flush()
        self.db.refresh(product)
        return product

    def get_by_id(self, product_id: int) -> Product | None:
        """Obtiene producto por ID"""
        return self.db.get(Product, product_id)

    def exists_by_sku(self, sku: str) -> bool:
        """Verifica si existe un producto con ese SKU"""
        stmt = select(Product).where(Product.sku == sku)
        return self.db.execute(stmt).scalar_one_or_none() is not None
```

### Estructura de Proyecto

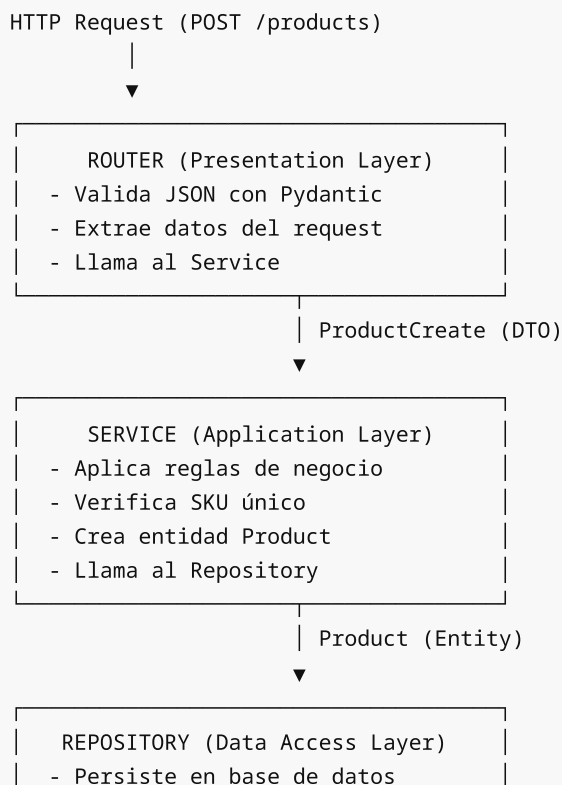
```
src/
├─ main.py           # Punto de entrada FastAPI
├─ config.py         # Configuración
├─ database.py       # Conexión DB
├─
├─ models/           # 📦 DATA LAYER - Entidades SQLAlchemy
│   └─ __init__.py
│   └─ base.py
│   └─ product.py
```

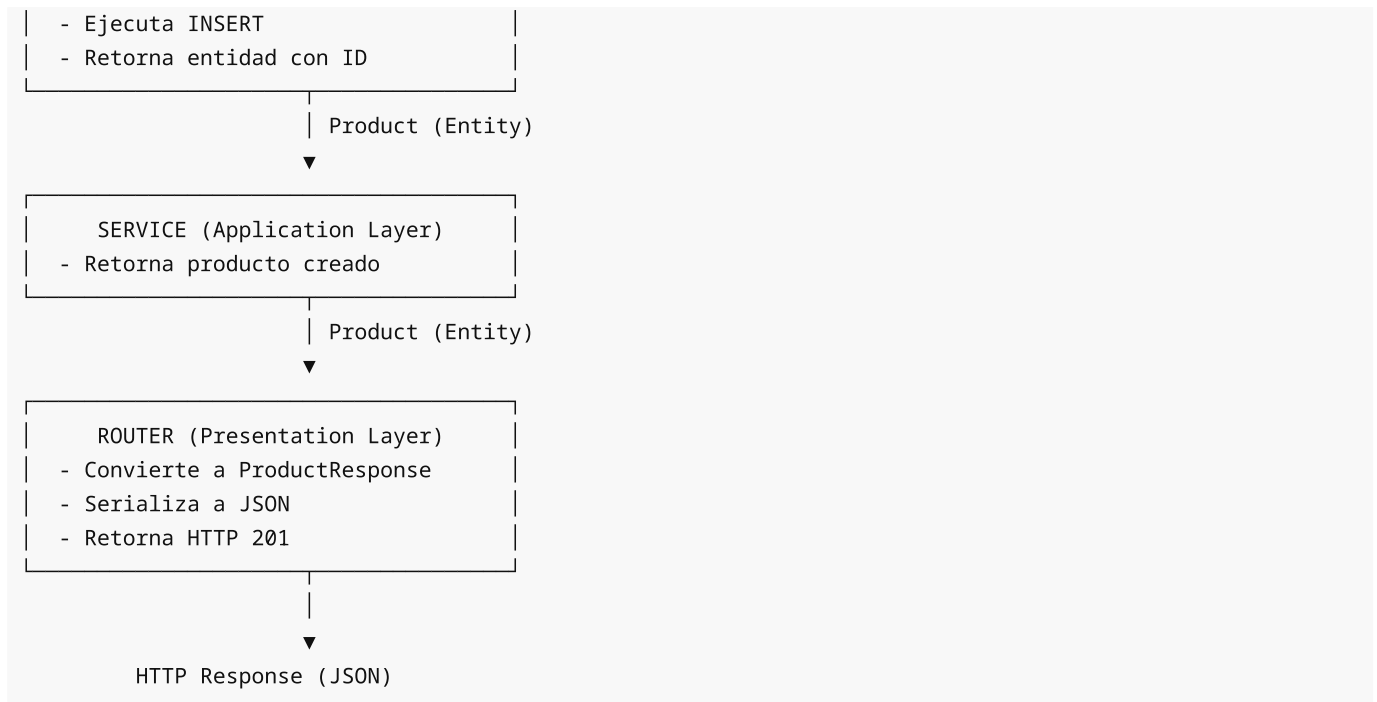
```

├── category.py
├── repositories/           # 🗄️ DATA LAYER - Acceso a datos
│   ├── __init__.py
│   ├── base.py
│   ├── product.py
│   └── category.py
├── schemas/               # 📄 PRESENTATION - DTOs Pydantic
│   ├── __init__.py
│   ├── product.py
│   └── category.py
├── services/              # ⚙️ APPLICATION - Lógica de negocio
│   ├── __init__.py
│   ├── product.py
│   └── category.py
├── routers/               # 🌐 PRESENTATION - Endpoints HTTP
│   ├── __init__.py
│   ├── products.py
│   └── categories.py
├── exceptions/            # ❌ Excepciones personalizadas
│   ├── __init__.py
│   ├── base.py
│   └── product.py
└── dependencies.py         # 🔗 Inyección de dependencias

```

## 🔄 Flujo de una Request





## ✓ Beneficios de la Arquitectura en Capas

| Beneficio             | Descripción                                    |
|-----------------------|------------------------------------------------|
| <b>Mantenibilidad</b> | Cambios aislados en una capa                   |
| <b>Testabilidad</b>   | Cada capa se puede testear independientemente  |
| <b>Reusabilidad</b>   | Services reutilizables en diferentes contextos |
| <b>Escalabilidad</b>  | Equipos pueden trabajar en paralelo            |
| <b>Comprensión</b>    | Estructura predecible y familiar               |

## ⚠ Errores Comunes

### ✗ Lógica de negocio en Router

```
# ✗ MAL - lógica en router
@router.post("/products/")
def create_product(data: ProductCreate, db: Session = Depends(get_db)):
    # Esto debería estar en el Service
    if db.query(Product).filter(Product.sku == data.sku).first():
        raise HTTPException(status_code=409, detail="SKU exists")

    product = Product(**data.model_dump())
    db.add(product)
    db.commit()
    return product
```

### ✓ Lógica en Service

```
# ✅ BIEN - router delega al service
@router.post("/products/")
def create_product(
    data: ProductCreate,
    service: ProductService = Depends(get_product_service)
):
    try:
        return service.create(data)
    except ProductAlreadyExistsError as e:
        raise HTTPException(status_code=409, detail=str(e))
```

## Recursos

- [Clean Architecture - Robert C. Martin](#)
- [Layered Architecture - Martin Fowler](#)

## ✅ Checklist

- ☐ Entiendo las tres capas principales
- ☐ Sé qué responsabilidades tiene cada capa
- ☐ Puedo organizar un proyecto en capas
- ☐ Entiendo el flujo de datos entre capas

## DTOs y Mappers

 DTOs y Conversiones

## Objetivos

- Entender qué son los DTOs y por qué usarlos
- Implementar DTOs específicos por operación
- Crear Mappers para convertir entre capas
- Separar modelos de DB de schemas de API

## ¿Qué es un DTO?

**DTO (Data Transfer Object)** es un objeto que transporta datos entre capas o procesos. Su único propósito es contener datos, sin lógica de negocio.

### ¿Por qué usar DTOs?

```
# ❌ SIN DTOs - Exponiendo modelo de DB directamente
@router.get("/users/{id}")
def get_user(id: int, db: Session = Depends(get_db)):
    user = db.get(User, id)
    return user # Expone password_hash, campos internos, etc.

# ✅ CON DTOs - Control sobre qué se expone
@router.get("/users/{id}", response_model=UserResponse)
```

```
def get_user(id: int, service: UserService = Depends(get_service)):
    return service.get_by_id(id) # Retorna DTO sin campos sensibles
```

## Tipos de DTOs

### 1. Request DTOs (Input)

```
# schemas/product.py
from pydantic import BaseModel, Field

class ProductCreate(BaseModel):
    """DTO para CREAR producto - todos los campos requeridos"""
    name: str = Field(..., min_length=1, max_length=200)
    sku: str = Field(..., pattern=r"^[A-Z]{3}-\d{4}$")
    price: float = Field(..., gt=0)
    stock: int = Field(default=0, ge=0)
    category_id: int

class ProductUpdate(BaseModel):
    """DTO para ACTUALIZAR producto - campos opcionales"""
    name: str | None = Field(None, min_length=1, max_length=200)
    price: float | None = Field(None, gt=0)
    stock: int | None = Field(None, ge=0)
    category_id: int | None = None
    # SKU no se puede cambiar (regla de negocio)
```

### 2. Response DTOs (Output)

```
from datetime import datetime
from pydantic import ConfigDict

class ProductResponse(BaseModel):
    """DTO de respuesta - lo que ve el cliente"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    sku: str
    price: float
    stock: int
    category_id: int
    created_at: datetime
    # NO incluye: updated_at interno, soft_delete flags, etc.

class ProductDetail(ProductResponse):
    """DTO con más detalle - incluye relaciones"""
    category: "CategoryResponse"
```

```
class ProductList(BaseModel):
    """DTO para listados con paginación"""
    items: list[ProductResponse]
    total: int
    page: int
    page_size: int
    pages: int
```

### 3. Internal DTOs (Entre capas)

```
class ProductFilter(BaseModel):
    """DTO para filtros internos"""
    category_id: int | None = None
    min_price: float | None = None
    max_price: float | None = None
    in_stock: bool | None = None
    search: str | None = None
```

## Mappers

Los **Mappers** convierten entre diferentes representaciones de datos.

### Mapper Simple (Funciones)

```
# mappers/product.py
from models.product import Product
from schemas.product import ProductCreate, ProductResponse

def to_entity(dto: ProductCreate) -> Product:
    """Convierte DTO → Entity"""
    return Product(
        name=dto.name,
        sku=dto.sku,
        price=dto.price,
        stock=dto.stock,
        category_id=dto.category_id
    )

def to_response(entity: Product) -> ProductResponse:
    """Convierte Entity → Response DTO"""
    return ProductResponse(
        id=entity.id,
        name=entity.name,
        sku=entity.sku,
        price=entity.price,
        stock=entity.stock,
        category_id=entity.category_id,
        created_at=entity.created_at
    )
```



```
def to_response_list(entities: list[Product]) -> list[ProductResponse]:
    """Convierte lista de entities"""
    return [to_response(e) for e in entities]
```

## Mapper como Clase

```
class ProductMapper:
    """Centraliza conversiones de Product"""

    @staticmethod
    def to_entity(dto: ProductCreate) -> Product:
        return Product(
            name=dto.name,
            sku=dto.sku,
            price=dto.price,
            stock=dto.stock,
            category_id=dto.category_id
        )

    @staticmethod
    def to_response(entity: Product) -> ProductResponse:
        return ProductResponse.model_validate(entity)

    @staticmethod
    def update_entity(entity: Product, dto: ProductUpdate) -> Product:
        """Aplica cambios del DTO a la entidad"""
        update_data = dto.model_dump(exclude_unset=True)
        for field, value in update_data.items():
            setattr(entity, field, value)
        return entity
```

## Uso en Service

```
# services/product.py
from schemas.product import ProductCreate, ProductUpdate, ProductResponse
from mappers.product import ProductMapper

class ProductService:
    def __init__(self, repo: ProductRepository):
        self.repo = repo

    def create(self, data: ProductCreate) -> ProductResponse:
        # Validaciones de negocio
        if self.repo.exists_by_sku(data.sku):
            raise ProductAlreadyExistsError(f"SKU '{data.sku}' exists")

        # DTO -> Entity
        entity = ProductMapper.to_entity(data)

        # Persistir
        saved = self.repo.add(entity)
```

```

# Entity → Response DTO
return ProductMapper.to_response(saved)

def update(self, product_id: int, data: ProductUpdate) -> ProductResponse:
    entity = self.repo.get_by_id(product_id)
    if not entity:
        raise ProductNotFoundError(f"Product {product_id} not found")

    # Aplicar cambios
    updated = ProductMapper.update_entity(entity, data)
    saved = self.repo.update(updated)

    return ProductMapper.to_response(saved)

```

## Comparación: Con y Sin DTOs

### Sin DTOs (Modelo expuesto)

```

# models/user.py
class User(Base):
    id: Mapped[int]
    email: Mapped[str]
    password_hash: Mapped[str] # ⚠ Sensible
    is_admin: Mapped[bool]      # ⚠ Interno
    login_attempts: Mapped[int] # ⚠ Interno
    created_at: Mapped[datetime]

# ❌ Expone todo al cliente
@router.get("/users/{id}")
def get_user(id: int):
    return db.get(User, id) # Incluye password_hash!

```

### Con DTOs (Control total)

```

# schemas/user.py
class UserResponse(BaseModel):
    """Solo campos públicos"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    email: str
    created_at: datetime
    # NO: password_hash, is_admin, login_attempts

# ✅ Solo datos públicos
@router.get("/users/{id}", response_model=UserResponse)
def get_user(id: int, service: UserService = Depends()):
    return service.get_by_id(id)

```

## Pydantic model\_validate

Pydantic v2 facilita la conversión con `model_validate` :

```

from pydantic import BaseModel, ConfigDict

class ProductResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    price: float

# Convierte automáticamente SQLAlchemy → Pydantic
product_entity = repo.get_by_id(1)
response = ProductResponse.model_validate(product_entity)

```

## ✓ Mejores Prácticas

1. **Un DTO por operación:** Create , Update , Response
2. **Response DTO siempre:** Nunca exponer modelos de DB
3. **Validaciones en DTOs:** Usar Field() de Pydantic
4. **Mappers centralizados:** Fácil de mantener y testear
5. **from\_attributes=True:** Facilita conversión ORM → DTO

## ✓ Checklist

- ☐ Entiendo la diferencia entre DTO y Entity
- ☐ Sé crear DTOs para Create, Update y Response
- ☐ Puedo implementar Mappers
- ☐ Entiendo cuándo usar `model_validate`

## Manejo de Errores por Capas

 Jerarquía de Excepciones

## Objetivos

- Diseñar excepciones personalizadas por capa
- Implementar traducción de errores entre capas
- Crear exception handlers centralizados
- Producir respuestas de error consistentes

## El Problema

Sin una estrategia clara, los errores se propagan de forma inconsistente:

```

# ✗ Problemas comunes
@router.post("/products/")
def create_product(data: ProductCreate, db: Session = Depends()):
    try:
        # Acceso directo a DB

```

```

        product = Product(**data.model_dump())
        db.add(product)
        db.commit()
    except IntegrityError:
        # Error técnico expuesto al cliente
        raise HTTPException(status_code=500, detail="IntegrityError")
    except Exception as e:
        # Error genérico, sin información útil
        raise HTTPException(status_code=500, detail=str(e))

```

## Jerarquía de Excepciones

### Base Exception

```

# exceptions/base.py
class AppException(Exception):
    """Base para todas las excepciones de la aplicación"""

    def __init__(self, message: str, code: str | None = None):
        self.message = message
        self.code = code or self.__class__.__name__
        super().__init__(message)

class NotFoundError(AppException):
    """Recurso no encontrado"""
    pass

class ConflictError(AppException):
    """Conflicto (duplicado, estado inválido)"""
    pass

class ValidationError(AppException):
    """Error de validación de negocio"""
    pass

class UnauthorizedError(AppException):
    """No autorizado"""
    pass

class ForbiddenError(AppException):
    """Sin permisos"""
    pass

```

### Excepciones de Dominio

```

# exceptions/product.py
from .base import NotFoundError, ConflictError, ValidationError

```

```

class ProductNotFoundError(NotFoundError):
    """Producto no encontrado"""

    def __init__(self, product_id: int):
        super().__init__(
            message=f"Product with id {product_id} not found",
            code="PRODUCT_NOT_FOUND"
        )

class ProductAlreadyExistsError(ConflictError):
    """SKU duplicado"""

    def __init__(self, sku: str):
        super().__init__(
            message=f"Product with SKU '{sku}' already exists",
            code="PRODUCT_SKU_DUPLICATE"
        )

class InsufficientStockError(ValidationError):
    """Stock insuficiente"""

    def __init__(self, product_id: int, requested: int, available: int):
        super().__init__(
            message=f"Insufficient stock for product {product_id}. "
                    f"Requested: {requested}, Available: {available}",
            code="INSUFFICIENT_STOCK"
        )

```

## Errores por Capa

### Data Access Layer (Repository)

```

# repositories/product.py
from sqlalchemy.exc import IntegrityError

class ProductRepository:
    def add(self, product: Product) -> Product:
        try:
            self.db.add(product)
            self.db.flush()
            self.db.refresh(product)
            return product
        except IntegrityError as e:
            self.db.rollback()
            # Traducir error técnico a error de dominio
            if "unique constraint" in str(e).lower():
                raise ProductAlreadyExistsError(product.sku)
            raise # Re-raise si es otro tipo de IntegrityError

```

## Application Layer (Service)

```
# services/product.py
class ProductService:
    def get_by_id(self, product_id: int) -> Product:
        product = self.repo.get_by_id(product_id)
        if not product:
            raise ProductNotFoundError(product_id)
        return product

    def reduce_stock(self, product_id: int, quantity: int) -> Product:
        product = self.get_by_id(product_id)

        # Validación de negocio
        if product.stock < quantity:
            raise InsufficientStockError(
                product_id=product_id,
                requested=quantity,
                available=product.stock
            )

        product.stock -= quantity
        return self.repo.update(product)
```

## Presentation Layer (Router)

```
# routers/products.py
from exceptions.product import (
    ProductNotFoundError,
    ProductAlreadyExistsError,
    InsufficientStockError
)

@router.post("/", response_model=ProductResponse, status_code=201)
def create_product(
    data: ProductCreate,
    service: ProductService = Depends(get_service)
):
    try:
        return service.create(data)
    except ProductAlreadyExistsError as e:
        raise HTTPException(status_code=409, detail=e.message)

@router.patch("/{product_id}/reduce-stock")
def reduce_stock(
    product_id: int,
    quantity: int,
    service: ProductService = Depends(get_service)
):
    try:
        return service.reduce_stock(product_id, quantity)
    except ProductNotFoundError as e:
```

```
        raise HTTPException(status_code=404, detail=e.message)
    except InsufficientStockError as e:
        raise HTTPException(status_code=400, detail=e.message)
```

## Exception Handlers Centralizados

En lugar de try/except en cada endpoint, usa handlers globales:

```
# main.py
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from exceptions.base import (
    AppException,
    NotFoundError,
    ConflictError,
    ValidationError
)

app = FastAPI()

# Schema para respuestas de error
class ErrorResponse(BaseModel):
    error: str
    code: str
    detail: str | None = None

@app.exception_handler(NotFoundError)
async def not_found_handler(request: Request, exc: NotFoundError):
    return JSONResponse(
        status_code=404,
        content={
            "error": "Not Found",
            "code": exc.code,
            "detail": exc.message
        }
    )

@app.exception_handler(ConflictError)
async def conflict_handler(request: Request, exc: ConflictError):
    return JSONResponse(
        status_code=409,
        content={
            "error": "Conflict",
            "code": exc.code,
            "detail": exc.message
        }
    )

@app.exception_handler(ValidationError)
async def validation_handler(request: Request, exc: ValidationError):
```

```

return JSONResponse(
    status_code=400,
    content={
        "error": "Validation Error",
        "code": exc.code,
        "detail": exc.message
    }
)

# Handler genérico para AppException no manejadas
@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=500,
        content={
            "error": "Internal Error",
            "code": exc.code,
            "detail": exc.message
        }
    )

```

## Router Simplificado

Con handlers globales, los routers son más limpios:

```

# routers/products.py
@router.post("/", response_model=ProductResponse, status_code=201)
def create_product(
    data: ProductCreate,
    service: ProductService = Depends(get_service)
):
    # Sin try/except - los handlers globales manejan errores
    return service.create(data)

@router.get("/{product_id}", response_model=ProductResponse)
def get_product(
    product_id: int,
    service: ProductService = Depends(get_service)
):
    return service.get_by_id(product_id)

```



## Mapecto de Excepciones → HTTP Status

| Excepción         | HTTP Status | Uso                           |
|-------------------|-------------|-------------------------------|
| NotFoundError     | 404         | Recurso no existe             |
| ConflictError     | 409         | Duplicado, estado inválido    |
| ValidationError   | 400         | Validación de negocio fallida |
| UnauthorizedError | 401         | No autenticado                |
| ForbiddenError    | 403         | Sin permisos                  |



|              |     |                |
|--------------|-----|----------------|
| AppException | 500 | Error genérico |
|--------------|-----|----------------|

## Respuestas de Error Consistentes

```
// 404 Not Found
{
  "error": "Not Found",
  "code": "PRODUCT_NOT_FOUND",
  "detail": "Product with id 123 not found"
}

// 409 Conflict
{
  "error": "Conflict",
  "code": "PRODUCT_SKU_DUPLICATE",
  "detail": "Product with SKU 'ABC-1234' already exists"
}

// 400 Bad Request
{
  "error": "Validation Error",
  "code": "INSUFFICIENT_STOCK",
  "detail": "Insufficient stock for product 5. Requested: 10, Available: 3"
}
```

## Checklist

- ☐ Tengo excepciones base para cada tipo de error
- ☐ Cada dominio tiene sus excepciones específicas
- ☐ Los repositorios traducen errores técnicos
- ☐ Los services lanzan excepciones de dominio
- ☐ Tengo exception handlers centralizados
- ☐ Las respuestas de error son consistentes

## Patrones Factory y Builder

 Dependency Injection

## Objetivos

- Entender el patrón Factory y cuándo usarlo
- Implementar el patrón Builder para objetos complejos
- Aplicar estos patrones en FastAPI
- Simplificar la creación de objetos en tests

## Patrón Factory

El patrón **Factory** encapsula la lógica de creación de objetos, centralizando y simplificando la instanciación.

## Problema

```
# ❌ Creación repetida y dispersa
# En router
product = Product(
    name=data.name,
    sku=data.sku,
    price=data.price,
    stock=data.stock,
    created_at=datetime.utcnow(),
    is_active=True
)

# En test
product = Product(
    name="Test Product",
    sku="TST-0001",
    price=99.99,
    stock=10,
    created_at=datetime.utcnow(),
    is_active=True
)

# En seed/fixture
product = Product(
    name="Seed Product",
    sku="SEED-001",
    price=50.0,
    stock=100,
    created_at=datetime.utcnow(),
    is_active=True
)
```

## Solución: Factory

```
# factories/product.py
from datetime import datetime
from models.product import Product
from schemas.product import ProductCreate

class ProductFactory:
    """Factory para crear instancias de Product"""

    @staticmethod
    def create(data: ProductCreate) -> Product:
        """Crea Product desde DTO"""
        return Product(
            name=data.name,
            sku=data.sku,
            price=data.price,
            stock=data.stock,
            created_at=datetime.utcnow(),
            is_active=True
        )
```

```

    )

    @staticmethod
    def create_for_test(
        name: str = "Test Product",
        sku: str | None = None,
        price: float = 99.99,
        stock: int = 10
    ) -> Product:
        """Crea Product para testing con valores por defecto"""
        import uuid
        return Product(
            name=name,
            sku=sku or f"TST-{uuid.uuid4().hex[:4].upper()}",
            price=price,
            stock=stock,
            created_at=datetime.utcnow(),
            is_active=True
        )

```

## Uso

```

# En Service
class ProductService:
    def create(self, data: ProductCreate) -> Product:
        product = ProductFactory.create(data)
        return self.repo.add(product)

# En Test
def test_product_service():
    product = ProductFactory.create_for_test(name="Widget", price=25.0)
    # ...

```

## Patrón Builder

El patrón **Builder** construye objetos complejos paso a paso, especialmente útil cuando hay muchos parámetros opcionales.

### Problema

```

# ❌ Constructor con muchos parámetros
order = Order(
    user_id=user.id,
    status=OrderStatus.PENDING,
    total=0,
    discount=0,
    tax=0,
    shipping_cost=0,
    shipping_address="123 Main St",
    billing_address="123 Main St",
    notes="Gift wrap please",
    created_at=datetime.utcnow(),
    items=[]
)

```

## Solución: Builder

```
# builders/order.py
from datetime import datetime
from models.order import Order, OrderItem, OrderStatus

class OrderBuilder:
    """Builder para construir Order paso a paso"""

    def __init__(self):
        self._user_id: int | None = None
        self._items: list[OrderItem] = []
        self._discount: float = 0
        self._shipping_address: str | None = None
        self._billing_address: str | None = None
        self._notes: str | None = None

    def for_user(self, user_id: int) -> "OrderBuilder":
        """Establece el usuario"""
        self._user_id = user_id
        return self

    def add_item(
        self,
        product_id: int,
        quantity: int,
        price: float
    ) -> "OrderBuilder":
        """Agrega un item al pedido"""
        item = OrderItem(
            product_id=product_id,
            quantity=quantity,
            unit_price=price,
            subtotal=quantity * price
        )
        self._items.append(item)
        return self

    def with_discount(self, discount: float) -> "OrderBuilder":
        """Aplica descuento"""
        self._discount = discount
        return self

    def ship_to(self, address: str) -> "OrderBuilder":
        """Establece dirección de envío"""
        self._shipping_address = address
        return self

    def bill_to(self, address: str) -> "OrderBuilder":
        """Establece dirección de facturación"""
        self._billing_address = address
        return self

    def with_notes(self, notes: str) -> "OrderBuilder":
```

```

        """Agrega notas"""
        self._notes = notes
        return self

def build(self) -> Order:
    """Construye el Order final"""
    if not self._user_id:
        raise ValueError("User ID is required")
    if not self._items:
        raise ValueError("At least one item is required")
    if not self._shipping_address:
        raise ValueError("Shipping address is required")

    # Calcular totales
    subtotal = sum(item.subtotal for item in self._items)
    tax = subtotal * 0.16 # 16% IVA
    shipping = 10.0 if subtotal < 100 else 0 # Envío gratis > $100
    total = subtotal - self._discount + tax + shipping

    return Order(
        user_id=self._user_id,
        status=OrderStatus.PENDING,
        subtotal=subtotal,
        discount=self._discount,
        tax=tax,
        shipping_cost=shipping,
        total=total,
        shipping_address=self._shipping_address,
        billing_address=self._billing_address or self._shipping_address,
        notes=self._notes,
        created_at=datetime.utcnow(),
        items=self._items
    )

```

## Uso

```

# Fluent interface - fácil de leer
order = (
    OrderBuilder()
    .for_user(user.id)
    .add_item(product_id=1, quantity=2, price=29.99)
    .add_item(product_id=3, quantity=1, price=49.99)
    .with_discount(10.0)
    .ship_to("123 Main St, City")
    .with_notes("Gift wrap please")
    .build()
)

```

## Factory para Dependencies

```

# factories/dependencies.py
from sqlalchemy.orm import Session

```

```

from repositories.product import ProductRepository
from repositories.order import OrderRepository
from services.product import ProductService
from services.order import OrderService
from unit_of_work import UnitOfWork

class ServiceFactory:
    """Factory para crear services con sus dependencias"""

    @staticmethod
    def create_product_service(db: Session) -> ProductService:
        repo = ProductRepository(db)
        return ProductService(repo)

    @staticmethod
    def create_order_service(db: Session) -> OrderService:
        uow = UnitOfWork(db)
        return OrderService(uow)

# dependencies.py
from fastapi import Depends
from database import get_db

def get_product_service(db: Session = Depends(get_db)) -> ProductService:
    return ServiceFactory.create_product_service(db)

def get_order_service(db: Session = Depends(get_db)) -> OrderService:
    return ServiceFactory.create_order_service(db)

```

## Factories para Testing

```

# tests/factories.py
from models.user import User
from models.product import Product
from models.order import Order

class TestDataFactory:
    """Factory para crear datos de prueba"""

    _user_counter = 0
    _product_counter = 0

    @classmethod
    def create_user(
        cls,
        username: str | None = None,
        email: str | None = None,
        is_active: bool = True
    ) -> User:

```

```

        cls._user_counter += 1
    return User(
        username=username or f"user_{cls._user_counter}",
        email=email or f"user_{cls._user_counter}@test.com",
        password_hash="hashed_password",
        is_active=is_active
    )

@classmethod
def create_product(
    cls,
    name: str | None = None,
    price: float = 99.99,
    stock: int = 10
) -> Product:
    cls._product_counter += 1
    return Product(
        name=name or f"Product {cls._product_counter}",
        sku=f"PRD-{cls._product_counter:04d}",
        price=price,
        stock=stock
    )

@classmethod
def reset_counters(cls):
    """Reset para entre tests"""
    cls._user_counter = 0
    cls._product_counter = 0

# Uso en tests
def test_order_creation():
    user = TestDataFactory.create_user()
    product1 = TestDataFactory.create_product(price=25.0)
    product2 = TestDataFactory.create_product(price=50.0)

    order = (
        OrderBuilder()
        .for_user(user.id)
        .add_item(product1.id, 2, product1.price)
        .add_item(product2.id, 1, product2.price)
        .ship_to("Test Address")
        .build()
    )

    assert order.total > 0

```

## Cuándo Usar Cada Patrón

| Patrón         | Usar cuando...                             |
|----------------|--------------------------------------------|
| <b>Factory</b> | Creación simple con variantes (test, prod) |
| <b>Builder</b> | Muchos parámetros opcionales               |

|                |                                         |
|----------------|-----------------------------------------|
| <b>Factory</b> | Encapsular lógica de instanciación      |
| <b>Builder</b> | Construcción paso a paso con validación |

## ✓ Checklist

- ☐ Sé crear Factories para entidades
- ☐ Puedo implementar Builder para objetos complejos
- ☐ Entiendo cuándo usar cada patrón
- ☐ Sé crear factories para testing

## Flujo de Datos Completo

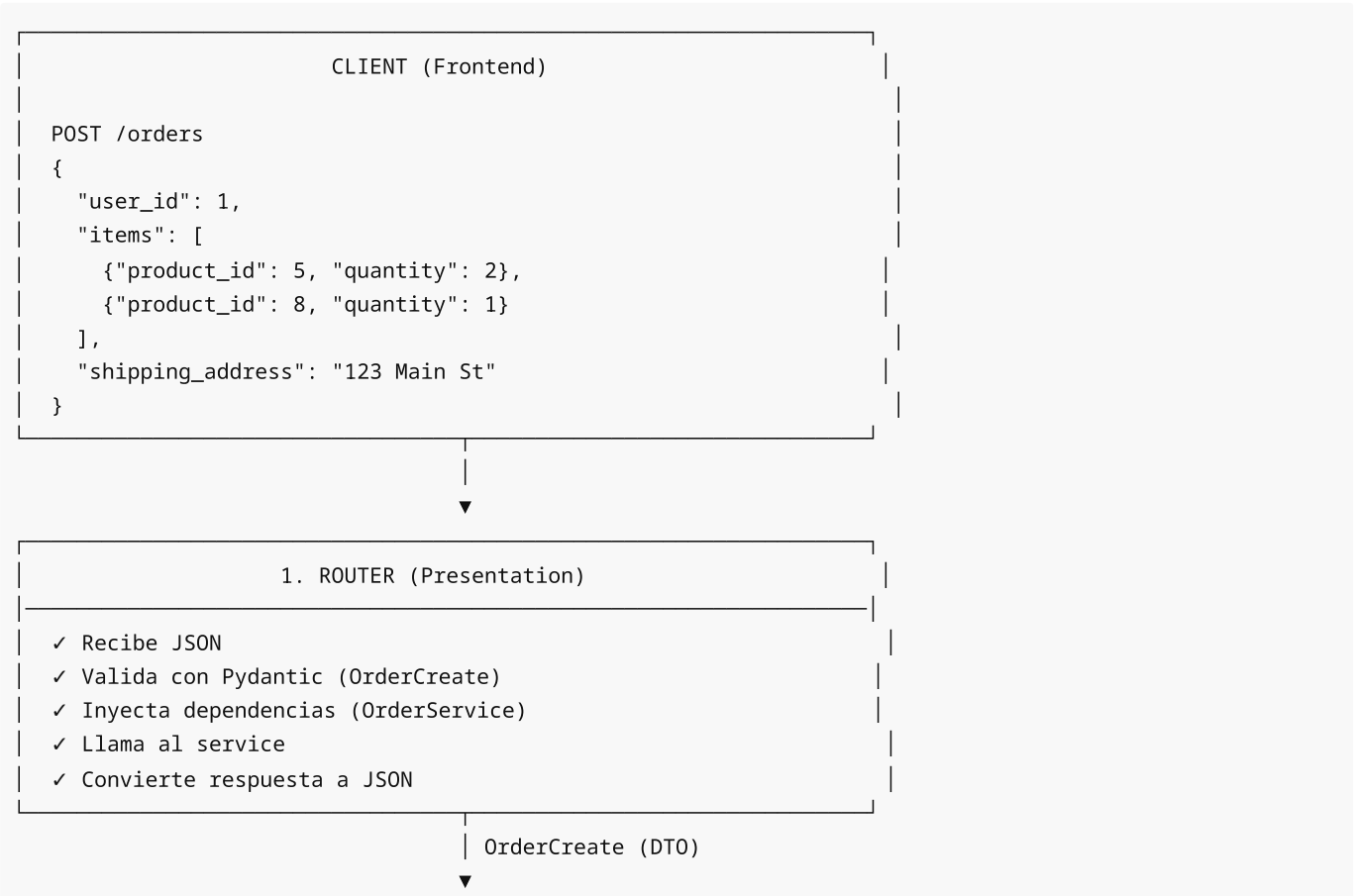
 Flujo de Datos

## Objetivos

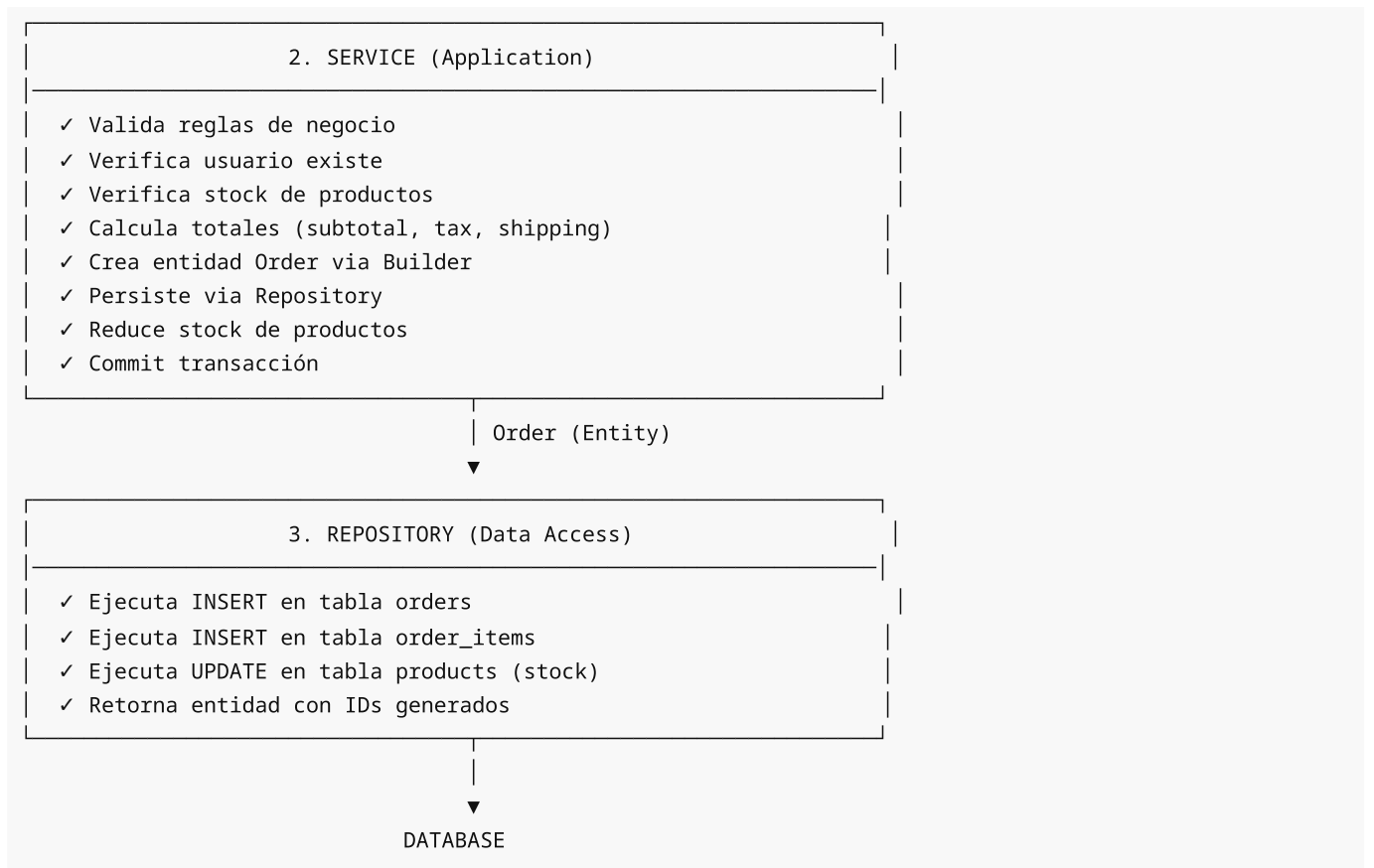
- Entender el flujo completo de una request
- Ver la transformación de datos en cada capa
- Identificar responsabilidades en cada paso
- Implementar un flujo end-to-end

## El Viaje de una Request

Veamos el flujo completo de `POST /orders` para crear un pedido:







## Implementación Paso a Paso

### 1. DTOs (Schemas)

```
# schemas/order.py
from pydantic import BaseModel, Field
from datetime import datetime

class OrderItemCreate(BaseModel):
    """Item del pedido - entrada"""
    product_id: int
    quantity: int = Field(..., gt=0)

class OrderCreate(BaseModel):
    """Crear pedido - entrada"""
    user_id: int
    items: list[OrderItemCreate] = Field(..., min_length=1)
    shipping_address: str = Field(..., min_length=10)
    notes: str | None = None

class OrderItemResponse(BaseModel):
    """Item del pedido - salida"""
    id: int
    product_id: int
    product_name: str
    quantity: int
```

```

    unit_price: float
    subtotal: float

class OrderResponse(BaseModel):
    """Pedido - salida"""
    model_config = ConfigDict(from_attributes=True)

    id: int
    user_id: int
    status: str
    items: list[OrderItemResponse]
    subtotal: float
    tax: float
    shipping_cost: float
    total: float
    shipping_address: str
    created_at: datetime

```

## 2. Router (Presentation Layer)

```

# routers/orders.py
from fastapi import APIRouter, Depends, status

from schemas.order import OrderCreate, OrderResponse
from services.order import OrderService
from dependencies import get_order_service

router = APIRouter(prefix="/orders", tags=["orders"])

@router.post(
    "/",
    response_model=OrderResponse,
    status_code=status.HTTP_201_CREATED
)
def create_order(
    data: OrderCreate,
    service: OrderService = Depends(get_order_service)
):
    """
    Crea un nuevo pedido.

    - Valida datos de entrada (Pydantic)
    - Delega lógica al Service
    - Retorna OrderResponse
    """
    return service.create_order(data)

```

## 3. Service (Application Layer)

```

# services/order.py
from datetime import datetime

```

```

from schemas.order import OrderCreate, OrderResponse
from models.order import Order, OrderItem, OrderStatus
from repositories.order import OrderRepository
from repositories.product import ProductRepository
from repositories.user import UserRepository
from unit_of_work import UnitOfWork
from exceptions.order import InsufficientStockError
from exceptions.user import UserNotFoundError
from exceptions.product import ProductNotFoundError


class OrderService:
    def __init__(self, uow: UnitOfWork):
        self.uow = uow

    def create_order(self, data: OrderCreate) -> Order:
        """
        Crea un pedido aplicando reglas de negocio.

        1. Valida usuario
        2. Valida productos y stock
        3. Construye el pedido
        4. Actualiza inventario
        5. Persiste todo en una transacción
        """
        # 1. Validar usuario existe
        user = self.uow.users.get_by_id(data.user_id)
        if not user:
            raise UserNotFoundError(data.user_id)

        # 2. Validar productos y stock
        order_items = []
        subtotal = 0

        for item_data in data.items:
            product = self.uow.products.get_by_id(item_data.product_id)
            if not product:
                raise ProductNotFoundError(item_data.product_id)

            if product.stock < item_data.quantity:
                raise InsufficientStockError(
                    product_id=product.id,
                    requested=item_data.quantity,
                    available=product.stock
                )

            # Crear item
            item = OrderItem(
                product_id=product.id,
                quantity=item_data.quantity,
                unit_price=product.price,
                subtotal=product.price * item_data.quantity
            )
            order_items.append(item)
            subtotal += item.subtotal

```

```

# 3. Calcular totales
tax = subtotal * 0.16
shipping = 0 if subtotal >= 100 else 10
total = subtotal + tax + shipping

# 4. Crear orden
order = Order(
    user_id=data.user_id,
    status=OrderStatus.PENDING,
    subtotal=subtotal,
    tax=tax,
    shipping_cost=shipping,
    total=total,
    shipping_address=data.shipping_address,
    notes=data.notes,
    items=order_items,
    created_at=datetime.utcnow()
)

# 5. Persistir
saved_order = self.uow.orders.add(order)

# 6. Reducir stock
for item_data in data.items:
    product = self.uow.products.get_by_id(item_data.product_id)
    product.stock -= item_data.quantity
    self.uow.products.update(product)

# 7. Commit transacción
self.uow.commit()

return saved_order

```

#### 4. Repository (Data Access Layer)

```

# repositories/order.py
from sqlalchemy import select
from sqlalchemy.orm import Session, joinedload

from repositories.base import BaseRepository
from models.order import Order

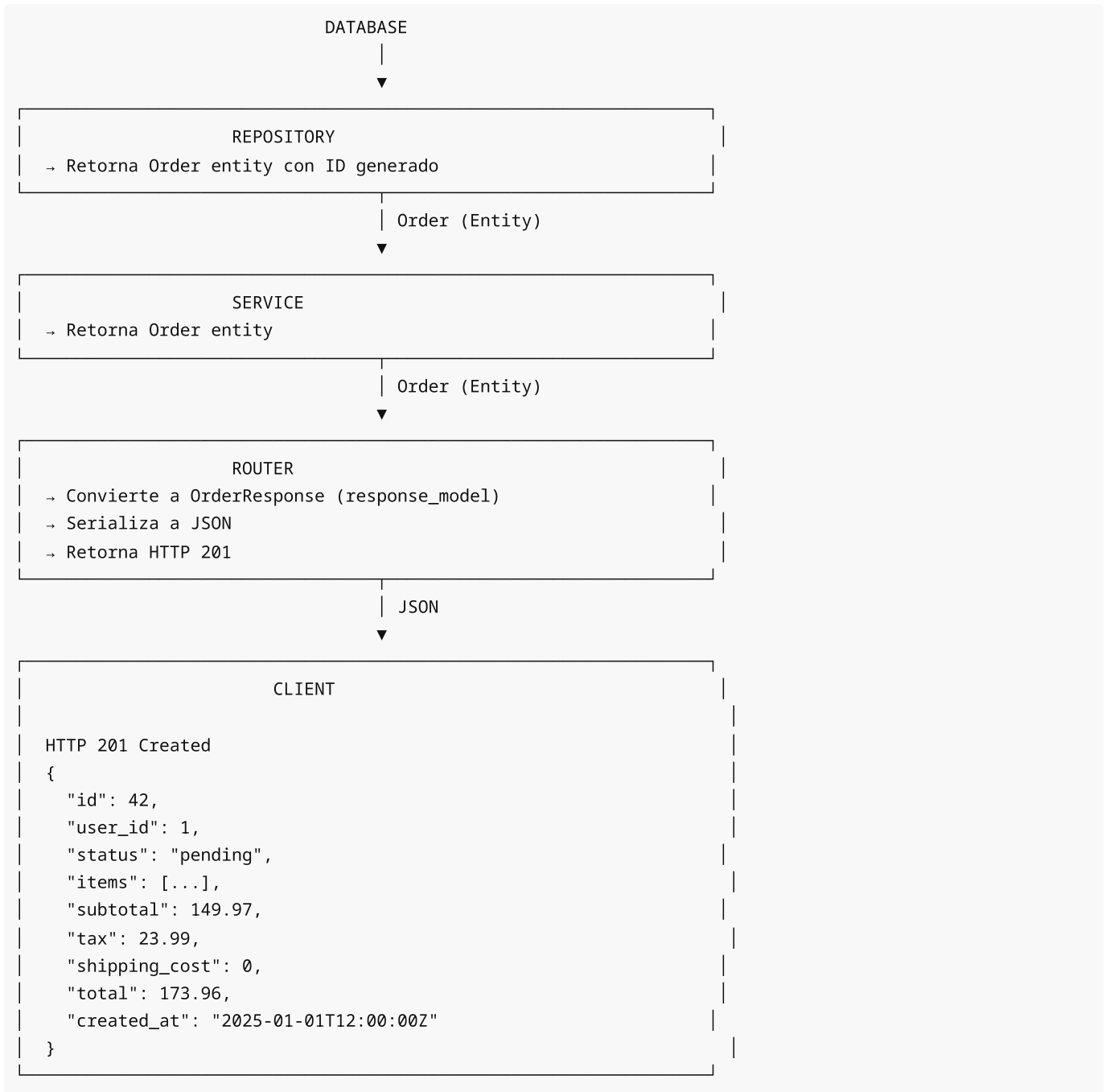
class OrderRepository(BaseRepository[Order]):
    def __init__(self, db: Session):
        super().__init__(db, Order)

    def get_by_id_with_items(self, order_id: int) -> Order | None:
        """Obtiene orden con items precargados"""
        stmt = (
            select(Order)
            .options(joinedload(Order.items))
            .where(Order.id == order_id)
        )
        return self.db.execute(stmt).unique().scalar_one_or_none()

```

```
def get_by_user(self, user_id: int) -> list[Order]:
    """Obtiene órdenes de un usuario"""
    stmt = select(Order).where(Order.user_id == user_id)
    return list(self.db.execute(stmt).scalars().all())
```

## El Viaje de la Response



## Resumen de Transformaciones

| Paso             | Input | Output            |
|------------------|-------|-------------------|
| Cliente → Router | JSON  | OrderCreate (DTO) |

|                      |                       |                |
|----------------------|-----------------------|----------------|
| Router → Service     | OrderCreate           | Order (Entity) |
| Service → Repository | Order                 | Order (con ID) |
| Repository → Service | Order                 | Order          |
| Service → Router     | Order                 | Order          |
| Router → Cliente     | Order → OrderResponse | JSON           |

## ✓ Checklist

- ☐ Entiendo el flujo completo de request a response
- ☐ Sé qué transformaciones ocurren en cada capa
- ☐ Puedo identificar responsabilidades por capa
- ☐ Entiendo cómo fluyen los datos entre capas

## 🏗️ Práctica 01: Estructura de Capas

### 🎯 Objetivos

- Crear la estructura de carpetas para arquitectura en capas
- Implementar las tres capas: Presentation, Application, Data Access
- Configurar las dependencias entre capas
- Verificar el flujo correcto de datos

### 📋 Descripción

En esta práctica crearás la estructura base de un proyecto con arquitectura en capas completa. Implementarás un CRUD simple de `Category` siguiendo el patrón **Router → Service → Repository**.

### 📁 Estructura del Proyecto

```
starter/
├─ main.py           # Punto de entrada
├─ config.py         # Configuración
├─ database.py       # Conexión a DB
├─ models/
│   └─ __init__.py
│   └─ category.py   # Entity Category
├─ schemas/
│   └─ __init__.py
│   └─ category.py   # DTOs Category
├─ repositories/
│   └─ __init__.py
│   └─ base.py       # Repository base
│   └─ category.py   # CategoryRepository
├─ services/
│   └─ __init__.py
│   └─ category.py   # CategoryService
├─ routers/
│   └─ __init__.py
```

|  |                    |                             |
|--|--------------------|-----------------------------|
|  | └─ categories.py   | # Endpoints                 |
|  | └─ dependencies.py | # Inyección de dependencias |

## Pasos

### Paso 1: Configuración Base

Abre `starter/config.py` y descomenta el código de configuración con Pydantic Settings.

### Paso 2: Modelo de Datos

Abre `starter/models/category.py` y descomenta el modelo SQLAlchemy.

### Paso 3: DTOs (Schemas)

Abre `starter/schemas/category.py` y descomenta los schemas Pydantic:

- `CategoryCreate` - para crear
- `CategoryUpdate` - para actualizar
- `CategoryResponse` - para respuestas

### Paso 4: Repository

Abre `starter/repositories/category.py` y descomenta:

- La clase `CategoryRepository`
- Los métodos CRUD básicos

### Paso 5: Service

Abre `starter/services/category.py` y descomenta:

- La clase `CategoryService`
- La lógica de negocio

### Paso 6: Router

Abre `starter/routers/categories.py` y descomenta:

- Los endpoints CRUD
- La inyección de dependencias

### Paso 7: Dependencies

Abre `starter/dependencies.py` y descomenta las funciones de inyección.

### Paso 8: Main

Abre `starter/main.py` y descomenta la inclusión del router.

## Verificación

Ejecuta la aplicación:

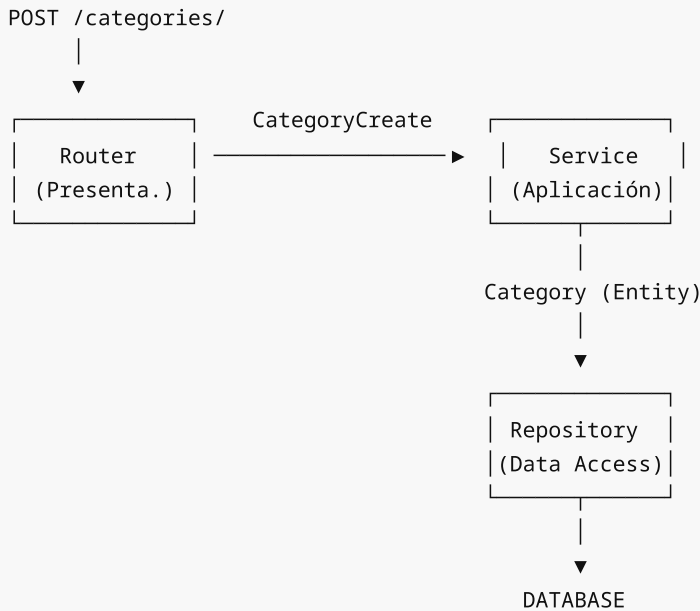
```
cd starter
uvicorn main:app --reload
```

Prueba en `http://localhost:8000/docs` :

1. **POST /categories/** - Crear categoría

2. **GET /categories/** - Listar categorías
3. **GET /categories/{id}** - Obtener una
4. **PATCH /categories/{id}** - Actualizar
5. **DELETE /categories/{id}** - Eliminar

## Flujo de Datos



## Resultado Esperado

Al completar la práctica tendrás:

-  Estructura de proyecto en capas
-  CRUD completo de Category
-  Separación clara de responsabilidades
-  Inyección de dependencias funcionando

## Práctica 02: DTOs y Conversiones

### Objetivos

- Crear DTOs específicos para cada operación
- Implementar Mappers para conversiones
- Usar `model_validate` de Pydantic v2
- Separar completamente Entity de Response

### Descripción

En esta práctica aprenderás a crear diferentes DTOs para entrada y salida, implementando Mappers que convierten entre entidades SQLAlchemy y schemas Pydantic.

Trabajarás con la entidad `Product` que tiene más complejidad que `Category`.



## Estructura

```
starter/
├─ main.py
├─ database.py
├─ models/
│   └─ product.py          # Entity Product
├─ schemas/
│   └─ product.py          # DTOs: Create, Update, Response, Detail
├─ mappers/
│   └─ product.py          # ProductMapper
├─ repositories/
│   └─ product.py          # ProductRepository
├─ services/
│   └─ product.py          # ProductService con Mapper
└─ routers/
    └─ products.py          # Endpoints
```

## Pasos

### Paso 1: Modelo Product

Abre `starter/models/product.py` y observa la entidad Product con campos que NO queremos exponer (como `cost_price`, `internal_notes`).

### Paso 2: DTOs Específicos

Abre `starter/schemas/product.py` y descomenta:

- `ProductCreate` - campos para crear (sin ID, sin timestamps)
- `ProductUpdate` - campos opcionales para PATCH
- `ProductResponse` - campos públicos (sin `cost_price`)
- `ProductDetail` - respuesta extendida con más info

### Paso 3: Mapper

Abre `starter/mappers/product.py` y descomenta:

- `to_entity()` - DTO → Entity
- `to_response()` - Entity → Response DTO
- `update_entity()` - Aplica Update DTO a Entity

### Paso 4: Service con Mapper

Abre `starter/services/product.py` y descomenta el uso del Mapper en cada operación.

### Paso 5: Router

Abre `starter/routers/products.py` y descomenta los endpoints.

## Flujo de Conversiones

```
POST /products/
|
|  JSON Input
▼
```



## ✓ Verificación

Ejecuta y prueba:

```
cd starter
uvicorn main:app --reload
```

1. **POST** `/products/` - Crea producto (no envíes `cost_price` en response)
2. **GET** `/products/{id}` - Verifica que `cost_price` NO aparece
3. **PATCH** `/products/{id}` - Actualiza solo algunos campos

## 🎯 Puntos Clave

- **ProductCreate**: Solo campos que el cliente puede enviar
- **ProductResponse**: Solo campos que el cliente puede ver
- **ProductUpdate**: Todos opcionales con `exclude_unset=True`
- **Mapper**: Centraliza toda la lógica de conversión

## ⚠ Práctica 03: Manejo de Errores por Capas

### 🎯 Objetivos

- Crear jerarquía de excepciones personalizadas
- Implementar exception handlers globales
- Traducir errores técnicos a errores de dominio

- Producir respuestas de error consistentes

---

## Descripción

En esta práctica implementarás un sistema completo de manejo de errores donde cada capa tiene sus propias excepciones y los handlers globales las traducen a respuestas HTTP apropiadas.

---

## Estructura

```
starter/
├─ main.py
├─ database.py
├─ exceptions/
│  ├─ __init__.py
│  ├─ base.py           # Excepciones base
│  └─ product.py       # Excepciones de Product
├─ handlers/
│  └─ exception_handlers.py # Handlers globales
├─ models/
│  └─ product.py
├─ schemas/
│  ├─ product.py
│  └─ error.py         # Schema de error
├─ repositories/
│  └─ product.py       # Traduce errores de DB
├─ services/
│  └─ product.py       # Lanza excepciones de dominio
└─ routers/
   └─ products.py      # Sin try/except (handlers globales)
```

---

## Pasos

### Paso 1: Excepciones Base

Abre `starter/exceptions/base.py` y descomenta la jerarquía de excepciones:

- `AppException` - Base
- `NotFoundError` - 404
- `ConflictError` - 409
- `ValidationError` - 400

### Paso 2: Excepciones de Dominio

Abre `starter/exceptions/product.py` y descomenta:

- `ProductNotFoundError`
- `ProductAlreadyExistsError`
- `InsufficientStockError`

### Paso 3: Schema de Error

Abre `starter/schemas/error.py` y descomenta `ErrorResponse`.

### Paso 4: Exception Handlers

Abre `starter/handlers/exception_handlers.py` y descomenta los handlers globales.

## Paso 5: Repository con Traducción

Abre `starter/repositories/product.py` y observa cómo traduce `IntegrityError` a excepciones de dominio.

## Paso 6: Service con Excepciones

Abre `starter/services/product.py` y observa cómo lanza excepciones específicas.

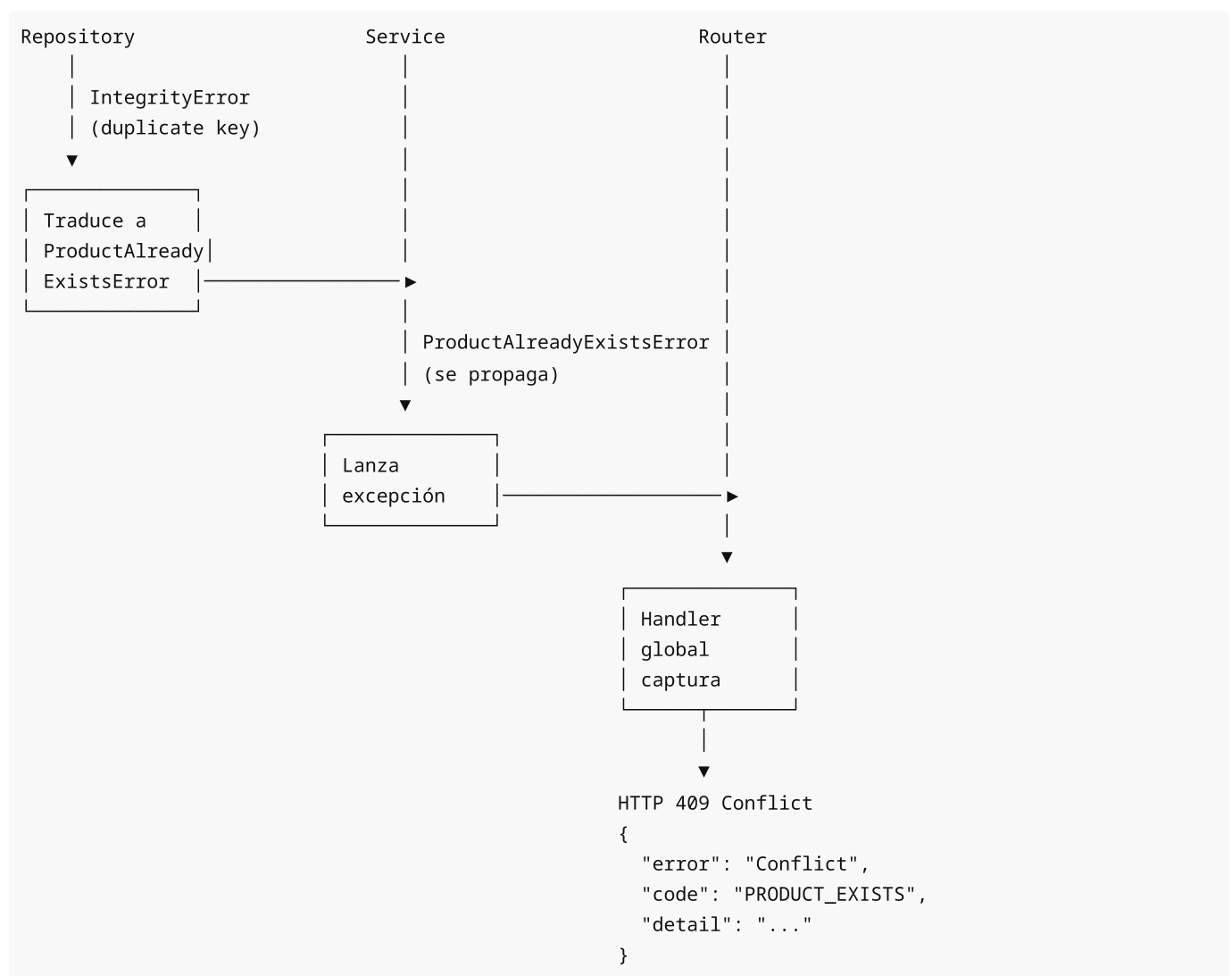
## Paso 7: Router Limpio

Abre `starter/routers/products.py` y observa cómo NO tiene try/except (los handlers globales manejan todo).

## Paso 8: Registrar Handlers

Abre `starter/main.py` y descomenta el registro de handlers.

## Flujo de Errores



## Verificación

```
cd starter
uvicorn main:app --reload
```

Prueba estos escenarios:

1. **GET /products/99999** → 404 con formato consistente
2. **POST /products/** (SKU duplicado) → 409 con código de error
3. **PATCH /products/{id}/reduce-stock** (sin stock) → 400

Todas las respuestas de error deben tener el mismo formato:

```
{
  "error": "Not Found",
  "code": "PRODUCT_NOT_FOUND",
  "detail": "Product with id 99999 not found"
}
```



## Práctica 04: Flujo Completo Request-Response



### Objetivos

- Implementar un flujo end-to-end completo
- Ver todas las capas trabajando juntas
- Entender las transformaciones en cada paso
- Crear una operación de negocio compleja



### Descripción

En esta práctica implementarás el flujo completo de **crear un pedido**, que involucra múltiples operaciones:

1. Validar usuario existe
2. Validar productos y stock
3. Calcular totales (subtotal, impuestos, envío)
4. Crear el pedido con sus items
5. Reducir stock de productos
6. Todo en una transacción



### Estructura

```
starter/
├─ main.py
├─ database.py
├─ models/
│   ├── user.py
│   ├── product.py
│   └─ order.py          # Order + OrderItem
├─ schemas/
│   ├── user.py
│   ├── product.py
│   └─ order.py          # OrderCreate, OrderResponse
├─ repositories/
│   ├── user.py
│   ├── product.py
│   └─ order.py
├─ services/
│   └─ order.py          # Lógica compleja
```

```
├─ exceptions/
│   ├── base.py
│   ├── user.py
│   ├── product.py
│   └── order.py
├─ handlers/
│   └── exception_handlers.py
└─ routers/
    └── orders.py
```

## Flujo de Crear Pedido

POST /orders/

```
{
  "user_id": 1,
  "items": [
    {"product_id": 5, "quantity": 2},
    {"product_id": 8, "quantity": 1}
  ],
  "shipping_address": "123 Main St"
}
```

|



### ROUTER

- Recibe JSON
- Valida con Pydantic (OrderCreate)
- Llama service.create\_order()

|



### SERVICE

1. Validar usuario existe
2. Por cada item:
  - Validar producto existe
  - Validar stock suficiente
  - Crear OrderItem
3. Calcular subtotal
4. Calcular tax (16%)
5. Calcular shipping
6. Crear Order
7. Reducir stock de productos
8. Persistir todo

|



### REPOSITORIES

- user\_repo.get\_by\_id()
- product\_repo.get\_by\_id()
- order\_repo.add()
- product\_repo.update() (stock)

|

## Pasos

### Paso 1: Modelos

Abre `starter/models/` y descomenta:

- `User` - usuario básico
- `Product` - producto con stock
- `Order` y `OrderItem` - pedido con relación

### Paso 2: Schemas

Abre `starter/schemas/order.py` y descomenta:

- `OrderItemCreate` - item de entrada
- `OrderCreate` - pedido de entrada
- `OrderItemResponse` - item de salida
- `OrderResponse` - pedido de salida

### Paso 3: Excepciones

Abre `starter/exceptions/` y descomenta todas las excepciones.

### Paso 4: Repositories

Abre `starter/repositories/` y descomenta los repositories.

### Paso 5: Order Service

Abre `starter/services/order.py` - **ESTE ES EL CORAZÓN.**

Descomenta y estudia `create_order()` :

- Cómo valida el usuario
- Cómo itera sobre items
- Cómo calcula totales
- Cómo maneja la transacción

### Paso 6: Router

Abre `starter/routers/orders.py` y descomenta el endpoint.

### Paso 7: Main

Descomenta todo en `main.py` .

## Verificación

```
cd starter
uvicorn main:app --reload
```

Crea datos de prueba primero (o usa el script `/seed`):

```
# Crear usuario
POST /users/
```

```
{ "name": "John", "email": "john@example.com" }

# Crear productos
POST /products/
{ "name": "Widget", "sku": "WDG-001", "price": 29.99, "stock": 10 }

POST /products/
{ "name": "Gadget", "sku": "GDG-001", "price": 49.99, "stock": 5 }
```

Luego crea el pedido:

```
POST /orders/
{
  "user_id": 1,
  "items": [
    { "product_id": 1, "quantity": 2 },
    { "product_id": 2, "quantity": 1 }
  ],
  "shipping_address": "123 Main St, City"
}
```

Response esperada:

```
{
  "id": 1,
  "user_id": 1,
  "status": "pending",
  "items": [
    {
      "id": 1,
      "product_id": 1,
      "product_name": "Widget",
      "quantity": 2,
      "unit_price": 29.99,
      "subtotal": 59.98
    },
    {
      "id": 2,
      "product_id": 2,
      "product_name": "Gadget",
      "quantity": 1,
      "unit_price": 49.99,
      "subtotal": 49.99
    }
  ],
  "subtotal": 109.97,
  "tax": 17.60,
  "shipping_cost": 0,
  "total": 127.57,
  "created_at": "2025-01-01T12:00:00Z"
}
```



- **Service orquesta:** Coordina múltiples repositorios
- **Transacción implícita:** Todo en una sesión
- **Validaciones de negocio:** En el service, no en el router
- **DTOs separados:** OrderCreate ≠ OrderResponse
- **Excepciones específicas:** UserNotFoundError, InsufficientStockError



## Proyecto Semana 08: API con Arquitectura MVC/Capas



### Tu Dominio Asignado

**Dominio:** [El instructor te asignará tu dominio único]

⚠️ **IMPORTANTE:** Cada aprendiz trabaja sobre un dominio diferente.



### Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que NO está en el pool. **Debes adaptar TODO a tu dominio asignado.**

| Concepto | Ejemplo Genérico | Adapta a tu Dominio |
|----------|------------------|---------------------|
| Entity   | Item             | {YourEntity}        |
| DTO      | ItemDTO          | {YourEntity}DTO     |
| Mapper   | ItemMapper       | {YourEntity}Mapper  |



### Objetivo

Implementar una **arquitectura MVC/Capas completa:**

- DTOs para transferencia de datos
- Mappers para conversión entre capas
- Exception handlers centralizados
- Separación clara de responsabilidades



### Requisitos Funcionales (Adapta a tu Dominio)

#### DTOs (Data Transfer Objects)

```
# Ejemplo genérico (Warehouse)
class ItemDTO(BaseModel):
    """DTO público - sin datos internos"""
    id: int
    sku: str
    name: str
    quantity: int
    zone_name: str # Denormalizado para el cliente

class ItemCreateDTO(BaseModel):
    """DTO para creación"""
    sku: str
    name: str
    quantity: int
    zone_id: int
```

```

class ItemInternalDTO(BaseModel):
    """DTO interno - incluye todo"""
    id: int
    sku: str
    name: str
    quantity: int
    zone_id: int
    created_by: str # Dato interno
    created_at: datetime

```

## Mappers

```

# Ejemplo genérico
class ItemMapper:
    @staticmethod
    def to_dto(entity: Item) -> ItemDTO:
        return ItemDTO(
            id=entity.id,
            sku=entity.sku,
            name=entity.name,
            quantity=entity.quantity,
            zone_name=entity.zone.name # Join
        )

    @staticmethod
    def to_entity(dto: ItemCreateDTO, user: str) -> Item:
        return Item(
            sku=dto.sku,
            name=dto.name,
            quantity=dto.quantity,
            zone_id=dto.zone_id,
            created_by=user
        )

    @staticmethod
    def to_dto_list(entities: list[Item]) -> list[ItemDTO]:
        return [ItemMapper.to_dto(e) for e in entities]

```

## Exception Handlers

```

# Ejemplo genérico
class ItemNotFoundError(Exception):
    def __init__(self, item_id: int):
        self.item_id = item_id
        super().__init__(f"Item {item_id} not found")

class DuplicateSKUError(Exception):
    def __init__(self, sku: str):
        self.sku = sku
        super().__init__(f"SKU {sku} already exists")

@app.exception_handler(ItemNotFoundError)
async def item_not_found_handler(request: Request, exc: ItemNotFoundError):

```

```
return JsonResponse(  
    status_code=404,  
    content={"error": "item_not_found", "item_id": exc.item_id}  
)
```

## Layer Flow

```
Request → Router → Service → Repository → DB  
                        ↓  
                    Mapper  
                        ↓  
Response ← DTO ← Service
```

## Estructura del Proyecto

```
starter/  
├─ main.py  
├─ api/  
│   ├── routers/  
│   ├── dtos/  
│   └── mappers/  
├─ services/  
├─ repositories/  
├─ domain/  
├─ exceptions/  
│   ├── handlers.py  
│   └── errors.py  
├─ pyproject.toml  
├─ Dockerfile  
└─ docker-compose.yml
```

## Criterios de Evaluación

| Criterio                           | Puntos |
|------------------------------------|--------|
| <b>Funcionalidad (40%)</b>         |        |
| DTOs bien diseñados                | 15     |
| Mappers funcionan                  | 15     |
| Exception handlers                 | 10     |
| <b>Adaptación al Dominio (35%)</b> |        |
| DTOs coherentes con negocio        | 12     |
| Excepciones específicas            | 13     |
| Originalidad (no copia ejemplo)    | 10     |
| <b>Calidad del Código (25%)</b>    |        |
| Separación de capas clara          | 10     |
| Flujo de datos correcto            | 10     |

|               |            |
|---------------|------------|
| Código limpio | 5          |
| <b>Total</b>  | <b>100</b> |

## ⚠ Política Anticopia

- ❌ **No copies** el ejemplo genérico "ItemDTO/ItemMapper"
- ✅ **Diseña** DTOs específicos de tu dominio
- ✅ **Crea** excepciones relevantes para tu negocio

## 📚 Recursos

- [DTO Pattern](#)
- [FastAPI Exception Handlers](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)

## 📖 Glosario - Semana 08

### Arquitectura en Capas Completa

#### A

**Application Layer (Capa de Aplicación)** Capa intermedia que contiene la lógica de negocio. Los Services orquestan operaciones entre múltiples repositories y aplican reglas de negocio.

```
class OrderService:
    def __init__(self, order_repo, user_repo, product_repo):
        self.order_repo = order_repo
        self.user_repo = user_repo
        self.product_repo = product_repo
```

#### B

**BaseRepository** Clase genérica que implementa operaciones CRUD comunes. Otros repositories heredan de ella para evitar duplicación de código.

```
class BaseRepository(Generic[T]):
    def get_by_id(self, entity_id: int) -> T | None:
        return self.db.get(self.model, entity_id)
```

**Business Logic (Lógica de Negocio)** Reglas y procesos específicos del dominio. Ejemplo: calcular impuestos, validar stock, aplicar descuentos. Vive en la capa de servicios.

#### C

**Capa (Layer)** División horizontal de responsabilidades en una aplicación. Cada capa tiene un propósito específico y se comunica solo con capas adyacentes.

**ConfigDict** Configuración de modelos Pydantic v2 que reemplaza la clase `Config` interna.

```
class UserResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

**Conflict Error** Error HTTP 409 que indica que la operación no puede completarse porque entra en conflicto con el estado actual del recurso (ej: email duplicado).

---

## D

**Data Access Layer (Capa de Acceso a Datos)** Capa responsable de la persistencia. Los Repositories encapsulan las operaciones con la base de datos.

**Data Transfer Object (DTO)** Objeto que transporta datos entre capas. En FastAPI, los schemas Pydantic actúan como DTOs.

```
class UserCreate(BaseModel): # DTO de entrada
    email: EmailStr
    password: str

class UserResponse(BaseModel): # DTO de salida
    id: int
    email: str
```

**Dependency Injection (DI)** Patrón donde las dependencias se proporcionan desde afuera en lugar de crearse internamente. FastAPI lo implementa con `Depends()`.

```
def get_user_service(
    repo: UserRepository = Depends(get_user_repository)
) -> UserService:
    return UserService(repo)
```

## E

**Entity (Entidad)** Objeto del dominio con identidad única, típicamente mapeado a una tabla de base de datos. En SQLAlchemy son los modelos ORM.

**Exception Handler** Función que intercepta excepciones y las convierte en respuestas HTTP apropiadas.

```
@app.exception_handler(AppException)
async def app_exception_handler(request, exc):
    return JSONResponse(status_code=exc.status_code, ...)
```

**exclude\_unset** Opción de Pydantic que excluye campos no proporcionados, útil para actualizaciones parciales (PATCH).

```
update_data = dto.model_dump(exclude_unset=True)
```

## F

**Factory Function** Función que crea y retorna instancias de objetos. En FastAPI se usan como dependencias para crear services y repositories.

```
def get_user_repository(db: Session = Depends(get_db)) -> UserRepository:
    return UserRepository(db)
```

**from\_attributes** Configuración de Pydantic que permite crear modelos desde objetos ORM (antes `orm_mode = True` ).

---

## G

**Generic Repository** Repository que usa genéricos de Python para ser reutilizable con diferentes tipos de entidades.

```
T = TypeVar("T")

class BaseRepository(Generic[T]):
    def __init__(self, db: Session, model: Type[T]):
        self.model = model
```

## H

**Handler** Función que maneja un evento o solicitud específica. En FastAPI, los exception handlers manejan errores globalmente.

---

## I

**IntegrityError** Excepción de SQLAlchemy cuando se viola una restricción de base de datos (unique, foreign key, etc.).

```
except IntegrityError:
    self.db.rollback()
    raise UserAlreadyExistsError(email)
```

## J

**joinedload** Estrategia de SQLAlchemy para cargar relaciones en una sola consulta SQL (eager loading).

```
stmt = select(Order).options(joinedload(Order.items))
```

## L

**Layered Architecture (Arquitectura en Capas)** Patrón arquitectónico que organiza el código en capas horizontales con responsabilidades específicas: Presentación → Aplicación → Datos.

---

## M

**Mapper** Clase que convierte entre diferentes representaciones de datos (Entity ↔ DTO).

```
class UserMapper:
    @staticmethod
    def to_entity(dto: UserCreate, password_hash: str) -> User:
        return User(email=dto.email, password_hash=password_hash, ...)

    @staticmethod
    def to_response(entity: User) -> UserResponse:
        return UserResponse.model_validate(entity)
```

**model\_validate** Método de Pydantic v2 para crear una instancia desde un objeto (reemplaza `from_orm()` ).

```
UserResponse.model_validate(user_entity)
```

---

## O

**Orchestration (Orquestación)** Coordinación de múltiples operaciones o servicios. Los Services orquestan repositories.

```
class OrderService:
    def create(self, data):
        user = self.user_repo.get_by_id(data.user_id) # Validar
        # ... validar productos, calcular totales, reducir stock
        return self.order_repo.add(order) # Guardar
```

## P

**Presentation Layer (Capa de Presentación)** Capa que maneja la interacción con el exterior (HTTP). Los Routers de FastAPI pertenecen a esta capa.

## R

**Repository Pattern** Patrón que abstrae la lógica de acceso a datos, proporcionando una interfaz de colección para las entidades.

```
class UserRepository:
    def get_by_id(self, user_id: int) -> User | None: ...
    def get_by_email(self, email: str) -> User | None: ...
    def add(self, user: User) -> User: ...
```

**Response Model** Schema Pydantic que define la estructura de la respuesta HTTP en FastAPI.

```
@router.get("/{user_id}", response_model=UserResponse)
def get_user(user_id: int): ...
```

## S

**Separation of Concerns (Separación de Responsabilidades)** Principio que establece que cada módulo debe tener una única responsabilidad. Base de la arquitectura en capas.

**Service Layer** Capa que contiene la lógica de negocio y orquesta operaciones entre repositories.

**Snapshot** Copia de datos en un momento específico. En pedidos, se guarda el nombre y precio del producto al momento de la compra.

```
class OrderItem(Base):
    product_name: Mapped[str] # Snapshot, no relación
    unit_price: Mapped[float] # Precio al momento de compra
```

## T

**Transaction (Transacción)** Conjunto de operaciones que deben completarse todas o ninguna. SQLAlchemy maneja transacciones automáticamente con `commit()` y `rollback()`.

## U

**Unit of Work** Patrón que mantiene una lista de objetos afectados por una transacción. La Session de SQLAlchemy implementa este patrón.

## V

**Validation Error** Error HTTP 422 que indica datos de entrada inválidos. Pydantic los genera automáticamente, pero también se pueden crear personalizados.

```
class InsufficientStockError(ValidationError):
    def __init__(self, product_id, requested, available):
        super().__init__(
            message=f"Insufficient stock...",
            error_code="INSUFFICIENT_STOCK"
        )
```



## Resumen de Capas

| Capa          | Componente        | Responsabilidad          |
|---------------|-------------------|--------------------------|
| Presentation  | Router            | HTTP, validación entrada |
| Application   | Service           | Lógica de negocio        |
| Data Access   | Repository        | Persistencia             |
| Cross-cutting | Exception Handler | Manejo de errores        |
| Cross-cutting | Mapper            | Conversión de datos      |



## Referencias

- [Martin Fowler - Patterns of Enterprise Architecture](#)
- [FastAPI - Bigger Applications](#)
- [Clean Architecture - Robert C. Martin](#)



# Semana 07: Arquitectura en Capas Completa



## Objetivos de la Semana

Al finalizar esta semana, serás capaz de:

- ☒ Consolidar la arquitectura Router → Service → Repository
- ☒ Implementar DTOs (Data Transfer Objects) para cada capa
- ☒ Aplicar el principio de separación de responsabilidades
- ☒ Manejar errores de forma estructurada por capas
- ☒ Crear factories y builders para objetos complejos
- ☒ Entender el flujo de datos entre capas



## Contenidos

### 1. Teoría

| Archivo                                  | Tema                       | Duración |
|------------------------------------------|----------------------------|----------|
| <a href="#">01-arquitectura-capas.md</a> | Arquitectura en capas MVC  | 25 min   |
| <a href="#">02-dtos-mappers.md</a>       | DTOs y Mappers entre capas | 25 min   |



|                                            |                                   |        |
|--------------------------------------------|-----------------------------------|--------|
| <a href="#">03-manejo-errores-capas.md</a> | Errores estructurados por capa    | 20 min |
| <a href="#">04-factories-builders.md</a>   | Patrones Factory y Builder        | 20 min |
| <a href="#">05-flujo-datos.md</a>          | Flujo completo Request → Response | 20 min |

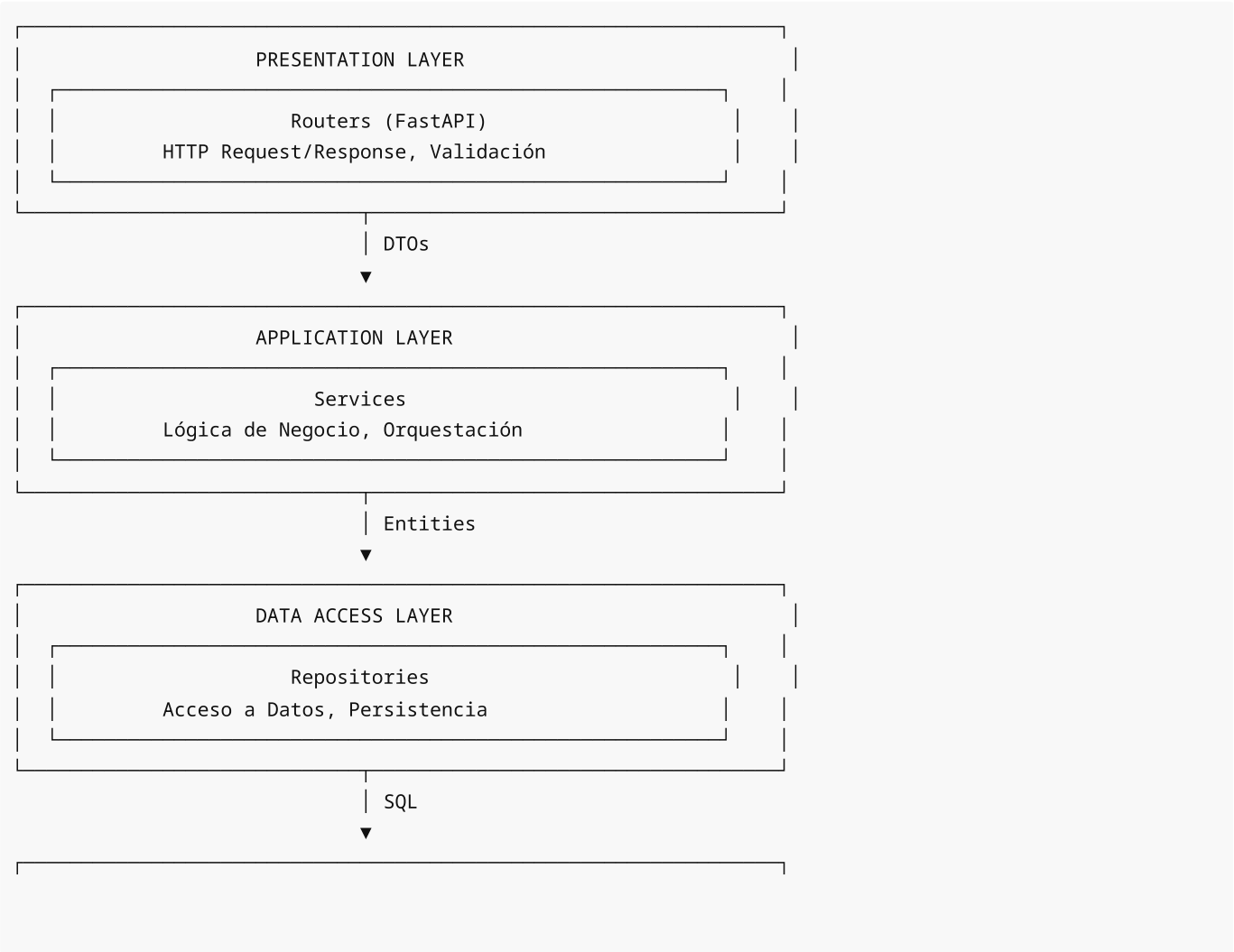
2. Prácticas

| Práctica                             | Tema                              | Duración |
|--------------------------------------|-----------------------------------|----------|
| <a href="#">01-estructura-capas</a>  | Organización de proyecto en capas | 30 min   |
| <a href="#">02-dtos-conversiones</a> | DTOs y conversiones               | 30 min   |
| <a href="#">03-error-handling</a>    | Manejo de errores por capa        | 30 min   |
| <a href="#">04-flujo-completo</a>    | Flujo Request → Response          | 30 min   |

3. Proyecto

| Proyecto                       | Descripción                            | Duración  |
|--------------------------------|----------------------------------------|-----------|
| <a href="#">E-Commerce API</a> | API completa con arquitectura en capas | 3-4 horas |

 Arquitectura de la Semana



## Requisitos Previos

- ☒ Week-05: SQLAlchemy básico
- ☒ Week-06: Service Layer
- ☒ Week-07: Repository Pattern y Unit of Work

## Distribución del Tiempo (6 horas)

| Actividad | Tiempo  |
|-----------|---------|
| Teoría    | 2 horas |
| Prácticas | 2 horas |
| Proyecto  | 2 horas |

## Entregable

Proyecto: [E-Commerce API](#)

API de e-commerce funcionando con arquitectura MVC completa:

- ☐ Presentation Layer (routers + schemas)
- ☐ Application Layer (services)
- ☐ Data Access Layer (repositories)
- ☐ DTOs y mappers implementados

## Conceptos Clave

- **Presentation Layer:** Maneja HTTP, serialización, validación de entrada
- **Application Layer:** Lógica de negocio, casos de uso, orquestación
- **Data Access Layer:** Persistencia, queries, transacciones
- **DTO (Data Transfer Object):** Objetos para transferir datos entre capas
- **Mapper:** Convierte entre DTOs y entidades

## Navegación

| Anterior                                        | Siguiente                                         |
|-------------------------------------------------|---------------------------------------------------|
| <a href="#">← Semana 07: Repository Pattern</a> | <a href="#">Semana 09: Ports &amp; Adapters →</a> |

## Rúbrica de Evaluación - Semana 08

### Arquitectura en Capas Completa

## Competencias a Evaluar

| Competencia                     | Peso |
|---------------------------------|------|
| Estructura de proyecto en capas | 25%  |
| DTOs y Mappers                  | 25%  |
| Manejo de errores por capa      | 25%  |
| Proyecto integrador             | 25%  |

---

## Criterios de Evaluación

### 1. Estructura de Proyecto en Capas (25%)

#### Excelente (90-100%)

- Separación clara: routers, services, repositories
- Dependencias fluyen en una dirección (hacia abajo)
- Cada capa tiene responsabilidad única
- Imports organizados y sin dependencias circulares

#### Satisfactorio (70-89%)

- Estructura de capas correcta con pequeñas mezclas
- Dependencias mayormente unidireccionales
- Responsabilidades claras en general

#### En Desarrollo (50-69%)

- Estructura parcial, algunas capas mezcladas
- Lógica de negocio en routers
- Acceso directo a DB desde services

#### Insuficiente (<50%)

- Sin separación de capas
- Todo en un archivo o sin organización
- Dependencias circulares

---

### 2. DTOs y Mappers (25%)

#### Excelente (90-100%)

- DTOs específicos para cada operación (Create, Update, Response)
- Mappers para convertir Entity ↔ DTO
- Validaciones en DTOs con Pydantic
- Separación clara entre schemas de API y modelos de DB

#### Satisfactorio (70-89%)

- DTOs implementados correctamente
- Conversiones manuales pero funcionales
- Validaciones básicas presentes

#### En Desarrollo (50-69%)

- DTOs parciales, reuso excesivo
- Sin mappers explícitos
- Exposición de modelos de DB en API

#### Insuficiente (<50%)

- Sin DTOs, modelos expuestos directamente
- Sin validaciones

- Mezcla de responsabilidades
- 

### 3. Manejo de Errores por Capa (25%)

#### Excelente (90-100%)

- Excepciones personalizadas por capa
- Traducción de errores entre capas
- Exception handlers centralizados
- Respuestas de error consistentes

#### Satisfactorio (70-89%)

- Excepciones personalizadas implementadas
- Manejo de errores funcional
- Respuestas de error estructuradas

#### En Desarrollo (50-69%)

- Excepciones genéricas
- Errores propagados sin traducir
- Respuestas inconsistentes

#### Insuficiente (<50%)

- Sin manejo de errores
  - Errores 500 sin información
  - Excepciones no controladas
- 

### 4. Proyecto Integrador (25%)

#### Excelente (90-100%)

- Todas las capas implementadas correctamente
- CRUD completo con DTOs
- Tests para services con fakes
- Documentación OpenAPI correcta
- Código limpio y bien organizado

#### Satisfactorio (70-89%)

- Capas implementadas funcionalmente
- CRUD operativo
- Algunos tests
- API documentada

#### En Desarrollo (50-69%)

- Implementación parcial
- CRUD incompleto
- Sin tests
- Documentación básica

#### Insuficiente (<50%)

- Proyecto no funcional
  - Sin estructura de capas
  - Sin documentación
- 



## Entregables

1. **Prácticas completadas** (4 ejercicios)
2. **Proyecto E-Commerce API** funcional

- 3. **Tests unitarios** para services
  - 4. **Documentación** de API (OpenAPI)
- 

## ✓ Checklist de Autoevaluación

### Estructura

- ☐ Carpetas separadas: routers/, services/, repositories/
- ☐ Schemas en carpeta propia
- ☐ Models separados de schemas
- ☐ Dependencies centralizadas

### DTOs

- ☐ CreatedDTO para creación
- ☐ UpdateDTO para actualización (campos opcionales)
- ☐ ResponseDTO para respuestas
- ☐ Mappers implementados

### Errores

- ☐ NotFoundError personalizado
- ☐ ValidationError personalizado
- ☐ ConflictError para duplicados
- ☐ Exception handlers configurados

### Proyecto

- ☐ API funcional con todos los endpoints
  - ☐ Tests ejecutándose sin errores
  - ☐ Documentación accesible en /docs
  - ☐ Código formateado y limpio
- 

17

## Fecha de Entrega

- **Prácticas:** Durante la semana
  - **Proyecto:** Fin de la semana 08
-